2026 International Conference on Intelligent and Innovative Technologies in Computing, Electrical and Electronics (IITCEE) | 979-8-3315-7746-9/26/$31.00 ©2026 IEEE | DOI: 10.1109/IITCEE67948.2026.11394602

Fusing Deterministic Caching and RAG-Enhanced LLM Inference for On-Device Conversational Command Execution

Girijamma H A
Dept. of Computer Science
RNS Institute of Technology
Bengaluru, India
girijamma.ha@rnsit.ac.in

Manasa K
Dept. of Computer Science
RNS Institute of Technology
Bengaluru, India
manasak2k3@gmail.com

Poorvi Srikanth
Dept. of Computer Science
RNS Institute of Technology
Bengaluru, India
spoorvi04@gmail.com

Maulik Dhananjaya
Dept. of Computer Science
RNS Institute of Technology
Bengaluru, India
maulik4002djay@gmail.com

Obulaneni Bharadwaj
Dept. of Computer Science
RNS Institute of Technology
Bengaluru, India
1rn22cs104.o.bharadwaj@gmail.com

***bstract*—The command-line interface (CLI) remains a pow- erful but often inaccessible tool for many users due to its steep learning curve and reliance on memorization of complex commands. We propose and implement JARVIS Shell, an AI- powered system companion that simplifies CLI usage by enabling natural language and voice-based interactions. The proposed sys- tem employs a multi-layered, hybrid architecture that combines a fast Command Cache, a robust Rule-Based Engine, and an intelligent Large Language Model (LLM) Inference layer. This layered approach ensures instantaneous responses for common tasks while providing deep, contextual understanding for complex or novel queries. A distinguishing element of the system is its SafeExecutor, which integrates a security and validation layer to prevent the execution of potentially dangerous commands. The system's effectiveness is demonstrated through its ability to translate and execute commands across different operating systems, interpret complex pipelines, and provide a safer, more intuitive user experience.**

***Index Terms*—Command Line Interface, Large Language Model, Natural language interface, shell automation, AI assistant, command execution, pipeline support.**

I. Introduction

Modern computing often requires users to interact with command-line interfaces (CLI) for system administration, software development, and automation tasks [1]. While CLIs offer unparalleled flexibility and efficiency, they are inherently difficult for novice users, who must memorize exact command syntax and flags. Mistyped commands can lead to errors or unintended consequences, creating a steep learning curve and limiting accessibility [2].

To address these challenges, we propose a hybrid, three-layered system that translates natural language queries into executable shell commands. The system combines a high-speed cache for common commands, a rule-based engine for predictable language patterns, and a locally-hosted Large Language Model (LLM) for complex or ambiguous queries [3]. This architecture balances performance, reliability, and expressive power, enabling both novice and experienced users to interact with the system naturally. By offloading frequent tasks to lightweight layers and reserving the LLM for com- plex queries, the system achieves low-latency responses while maintaining robustness and flexibility [4].

This work contributes a novel approach to bridging the gap between natural language and system-level command execution, offering a safer, more efficient, and more accessible interface for users across different expertise levels [5].

II. Background and Related Work

The command-line interface (CLI) remains one of the most powerful and efficient tools for interacting with a computer, offering unparalleled control and scriptability. However, its power is gated by a steep learning curve, requiring users to memorize a rigid, often non-intuitive syntax [6]. As human-computer interaction increasingly gravitates towards natural language, the gap between human intent and the CLI's syntactic demands presents a significant challenge. This chapter reviews the historical and contemporary approaches to bridging this gap, establishing the context and identifying the research opportunity that our work addresses [7].

A. Early Approaches: Rule-Based Natural Language Interfaces

The initial attempts to create conversational shells were predominantly rule-based, deterministic systems. These methods, while computationally efficient, were defined by their syntactic, rather than semantic, understanding of user input [8].

1) Direct Keyword Mapping: The most primitive approach involved static, one-to-one mapping of keywords to commands, typically implemented with a hash map. For instance,

979-8-3315-7746-9/26/$31.00 ©2026 IEEE

Authorized licensed use limited to: KNS Institute of Technology. Downloaded on March 03,2026 at 14:54:05 UTC from IEEE Xplore. Restrictions apply.

the keyword `list` would be directly translated to the `ls` command. While this method offers near-instantaneous execution with O(1) complexity, its utility is severely limited. It is inherently inflexible, unable to handle any arguments, options, or variations in phrasing, rendering it impractical for anything beyond the simplest of commands [9].

2) Regular Expression (Regex) Parsing: A more advanced technique utilized regular expressions to parse structured natural language patterns. A regex engine could identify and extract key entities from a query, such as a filename from the sentence "create a file named `report.txt`." This allowed for a degree of flexibility beyond simple keyword matching [10]. This inherent rigidity makes such systems prone to failure when faced with linguistic variance. As they operate on a purely syntactic level, they lack true semantic understanding. A minor deviation from the expected sentence structure, such as "make a file for me called `report.txt`," would cause the pattern to fail. Consequently, rule-based systems require extensive, handcrafted rules for every conceivable query and still fail to capture the nuance and variability of human language [11].

B. The Emergence of LLMs for Automated Command-Line Scripting

The recent advent of Large Language Models (LLMs), underpinned by the Transformer architecture, has fundamentally changed the landscape of human-computer interaction. These models have demonstrated a remarkable ability to understand semantic intent and construct well-formed, executable code based on natural language inputs. [12].

A seminal development in this area was OpenAI's Codex, the model that powered the initial release of GitHub Copilot. Codex proved that a sufficiently large model trained on a vast corpus of code could translate complex, high-level descriptions into functional code snippets. This marked a paradigm shift from brittle syntactic parsing to robust semantic interpretation [13].

More recently, the democratization of this technology through powerful, open-source models like Meta's Llama series and Microsoft's Phi-3 has made it possible to deploy capable AI reasoning engines locally. Tools like Ollama enable private, cost-effective, and offline command generation, removing the reliance on proprietary cloud APIs and addressing key privacy concerns. These advancements have unlocked the potential for a new generation of intelligent shell interfaces that were previously unfeasible [14].

C. The Research Gap: The Performance-Intelligence Dichotomy

The evolution of natural language shells reveals a persistent dichotomy between performance and intelligence.

- **Rule-Based Systems:** Characterized by low latency, high predictability, and computational efficiency. However, they are brittle, limited in scope, and require significant manual effort to create and maintain [15].
- **LLM-Only Systems:** Offer profound semantic understanding and flexibility, capable of handling novel and complex requests. Yet, these models are constrained by significant delays inherent to the computational cost of model inference and can introduce a degree of non-determinism [16].

This trade-off presents a clear research gap: the need for a blended architecture that cohesively integrates the advantages of both approaches. An ideal system would leverage the instantaneous performance of rule-based methods for common, simple tasks while reserving the powerful reasoning capabilities of an LLM for the complex "long-tail" of user queries [17]. The work presented in this paper proposes such a system, a three-layered architecture designed to resolve this performance-intelligence dichotomy and create a shell that is both fast and robust.

III. Methodology

Our methodology is based on a hybrid, multi-layered architecture designed to balance the speed of direct lookups with the power of advanced AI reasoning. The system processes user input through a cascading series of three distinct layers, ensuring that simple and common tasks are handled instantly while complex natural language queries are routed to the most intelligent processing engine.

A. Architectural Layer

The system architecture consists of the following distinct, cascading layers:

1) ***Layer 1: Command Cache (Instantaneous)****:*

- Technique: Simple dictionary lookup.
- Purpose: This layer serves as the first line of defense for efficiency. It is implemented as an in-memory hash map or a Python dictionary that stores the most com- mon and exact command inputs (e.g., `ls`, `pwd`, `cd ..`) mapped to their corresponding executable system calls. The performance gain is significant, as a dictionary lookup is an O(1) operation, which minimizes latency to provide an instant user experience. The cache can be pre-populated with a list of frequently used commands and can also be dynamically updated based on the user's command history to further optimize for individual usage patterns [18]. This layer effectively offloads a majority of the simple queries from the more complex downstream layers.

2) ***Layer 2: Rule-Based Engine (Fast)****:*

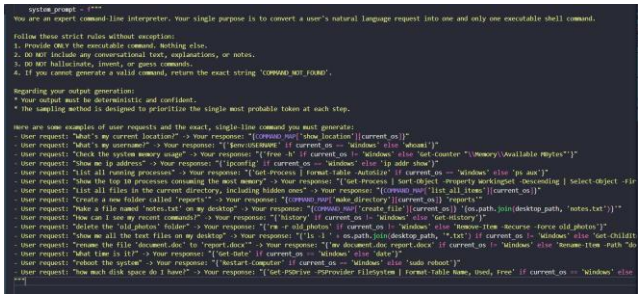
- Technique: Keyword and regular expression (regex) pattern matching.
- Purpose: This layer is designed to handle a wide range of simple, predictable natural language requests that follow predefined patterns without the computational overhead of the LLM. For example, a query like "list all files in the current directory" or "find files with a .txt extension" can be parsed using regex patterns to extract key entities (.txt) and map the query to a specific command (`ls *.txt`). This layer is built to be a highly efficient intermediary,

Authorized licensed use limited to: KNS Institute of Technology. Downloaded on March 03,2026 at 14:54:05 UTC from IEEE Xplore. Restrictions apply.

handling queries that are more complex than a direct cache lookup but not complex enough to require full AI reasoning [19]. By accurately categorizing and processing these requests, it significantly reduces the number of expensive LLM calls.

*3) **Layer 3: Reasoning Engine with Few-Shot Prompting**:* This layer serves as the core reasoning engine of the system, activated only when a user's query is too complex or novel for the deterministic initial layers. It is designed to interpret nuanced semantic intent and generate precise, executable commands [20]. The reliability of this layer is ensured through a combination of advanced prompt engineering techniques.

a) Core Technique: Few-Shot Prompting: The primary technique for guiding the LLM is Few-Shot Learning. Unlike zero-shot prompting (providing only an instruction), a few-shot prompt provides the model with a handful of high-quality, in-context examples of correct query-to-command translations [21].



```
system_prompt = f"""
You are an expert command-line interpreter. Your single purpose is to convert a user's natural language request into one and only one executable shell command.

Follow these strict rules without exception:
1. Provide ONLY the executable command. Nothing else.
2. DO NOT include any conversational text, explanations, or notes.
3. DO NOT hallucinate, invent, or guess commands.
4. If you cannot generate a valid command, return the exact string 'COMMAND_NOT_FOUND'.

Regarding your output generation:
* Your output must be deterministic and confident.
* The sampling method is designed to prioritize the single most probable token at each step.

Here are some examples of user requests and the exact, single-line command you must generate:
- User request: "what's my current location?" -> Your response: "{COMMAND_MAP['show_location'][current_os]}"
- User request: "what's my username?" -> Your response: "{$env:USERNAME' if current_os == 'Windows' else 'whoami'}"
- User request: "check the system memory usage" -> Your response: "{'free -h' if current_os != 'Windows' else 'Get-Counter "\\Memory\\Available MBytes"'}"
- User request: "show me ip address" -> Your response: "{'ipconfig' if current_os == 'Windows' else 'ip addr show'}"
- User request: "list all running processes" -> Your response: "{'Get-Process | Format-Table -AutoSize' if current_os == 'Windows' else 'ps aux'}"
- User request: "show the top 10 processes consuming the most memory" -> Your response: "{'Get-Process | Sort-Object -Property WorkingSet -Descending | Select-Object -Fir
- User request: "list all files in the current directory, including hidden ones" -> Your response: "{COMMAND_MAP['list_all_items'][current_os]}"
- User request: "create a new folder called 'reports'" -> Your response: "{COMMAND_MAP['make_directory'][current_os]} 'reports'"
- User request: "make a file named 'notes.txt' on my desktop" -> Your response: "{COMMAND_MAP['create_file'][current_os]} '{os.path.join(desktop_path, 'notes.txt')}'"
- User request: "how can I see my recent commands?" -> Your response: "{'history' if current_os != 'Windows' else 'Get-History'}"
- User request: "delete the 'old_photos' folder" -> Your response: "{'rm -r old_photos' if current_os != 'Windows' else 'Remove-Item -Recurse -Force old_photos'}"
- User request: "show me all the text files on my desktop" -> Your response: "{'ls -l ' + os.path.join(desktop_path, '*.txt') if current_os != 'Windows' else 'Get-ChildIt
- User request: "rename the file 'document.doc' to 'report.docx'" -> Your response: "{'mv document.doc report.docx' if current_os != 'Windows' else 'Rename-Item -Path "do
- User request: "what time is it?" -> Your response: "{'Get-Date' if current_os == 'Windows' else 'date'}"
- User request: "reboot the system" -> Your response: "{'Restart-Computer' if current_os == 'Windows' else 'sudo reboot'}"
- User request: "how much disk space do I have?" -> Your response: "{'Get-PSDrive -PSProvider FileSystem | Format-Table Name, Used, Free' if current_os == 'Windows' else
"""
```

Fig. 1. Few-Shot Prompting.

For this system, the prompt includes examples demonstrating:

- Cross-Platform Translation: Mapping `ls -a` (Linux) to `Get-ChildItem -Force` (PowerShell).
- Natural Language to Syntax: Translating "list folders on the desktop" into the correct `Get-ChildItem` command with the appropriate path.
- Pipeline Construction: Converting "list text files sorted by size" into a piped `Get-ChildItem | Sort-Object` command.

By observing these patterns, the model learns the required syntax, parameter usage, and output format without the need for fine-tuning, enabling it to generalize effectively to a wide variety of new, unseen queries.

b) Control via Parameter Tuning: To ensure the generated commands are both safe and predictable, we control the model's output by tuning its generation parameters. Specifically, we set a very low temperature (e.g., 0.1). Temperature governs the randomness of the model's output; a high temperature encourages creative or varied responses, whereas a low temperature makes the output more deterministic. For this application, a low temperature forces the model to select the most probable and syntactically correct command, acting as a critical guardrail against the generation of unexpected or potentially harmful instructions [22].

c) Model Selection and Local Execution: The system leverages a local instance of Microsoft's Phi-3 Mini, a state-of-the-art small language model, served via Ollama. This choice was deliberate, as it provides a strong balance between high performance and a compact footprint, allowing the shell to run effectively on consumer hardware. Local execution ensures that all user data remains private and enables full offline functionality, addressing key limitations of cloud-based API solutions [23].

*4) **Contextual Grounding: Retrieval-Augmented Generation (RAG)**:* To minimise hallucination, reduce latency, and provide answers tailored to the user's environment, the system integrates a **Retrieval-Augmented Generation (RAG)** module [30]. RAG introduces a fast, evidence-based memory between the reasoning engine and available knowledge, allowing the model to return accurate commands instead of recreating them from scratch [31].

a) Goals and High-Level Flow:

- **Accuracy:** Ground generated commands in authoritative sources such as manuals, shell histories, and configuration files [32].
- **Efficiency:** Supply concise evidence to the model, reducing token usage and response time [33].
- **Safety:** Filter unsafe or unverified material before it reaches the prompt [34].

The flow is as follows:

1) If the cache and rule-based layers cannot resolve a query, Layer 3 triggers the RAG retriever.
2) The retriever embeds the query, performs a similarity search in the vector store, and returns the most relevant passages [35].
3) These passages, along with few-shot examples, are assembled into the LLM prompt.
4) The model proposes a command, which is then validated by a safety layer before execution.

b) Corpus and Pre-processing: The RAG corpus is built from multiple knowledge sources:

- **Command history:** Frequently used commands and pipelines (e.g., `rsync --delete`).
- **Dotfiles & aliases:** Shell configuration such as `.bashrc`, `.zshrc`, and custom shortcuts [36].
- **Manuals and tutorials:** POSIX man pages, shell scripting books, and curated reference snippets.
- **Policies and blocklists:** Security rules to exclude dangerous instructions [37].

Documents are split into small, overlapping chunks (128–512 tokens) with metadata tags (origin, timestamp, and type) for traceability [38].

c) Embedding and Vector Store: Lightweight, high-quality embedding models ensure fast retrieval on consumer hardware. Embeddings are stored in an Approximate Nearest Neighbour (ANN) index such as FAISS, HNSW, Milvus, or Chroma [39]. Metadata (e.g., file path, source type) is preserved so the assistant can explain where each snippet originated.

Authorized licensed use limited to: KNS Institute of Technology. Downloaded on March 03,2026 at 14:54:05 UTC from IEEE Xplore. Restrictions apply.

d) Retrieval and Ranking: User queries are encoded and compared against the index. A hybrid ranking score combines:

- Semantic similarity between the query and stored chunks.
- Rule-based signals, such as regex matches for known commands [40].

Top candidates are cached for a short time to accelerate repeated queries.

e) Prompt Construction: Retrieved passages are summarised or truncated to respect the token budget. Few-shot examples (from Section 3) guide the model's output style [41]. The final prompt is assembled as:

instructions → retrieved context → examples → user question

f) Safe Execution: Before any suggestion is executed, both retrieved material and generated commands are scanned for dangerous operations (e.g., `rm -rf /`). Only validated commands reach the shell [42].

g) Integration with the Three-Layer Engine: RAG enhances Layer 3 as follows:

1) Calls the retriever when no cached or rule-based answer is available.
2) If confidence is very high, returns the retrieved command directly [43].
3) Otherwise, appends context to the prompt for the LLM.
4) Validates the LLM result via the SafeExecutor.

h) Operational Considerations:

- **Latency:** Compact embeddings and ANN search keep retrieval under ∼100 ms.
- **Privacy:** If the corpus includes sensitive files, embeddings remain local or are encrypted [44].
- **Adaptation:** New commands and documentation are continuously ingested, while old data is pruned using TTL or LRU policies.
- **Explainability:** Retrieved snippets can optionally be shown to help users understand the reasoning behind a recommendation [45].

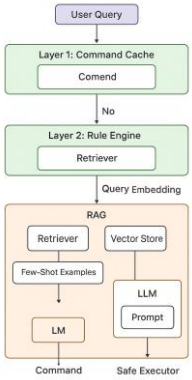


Fig. 2. Integration of RAG into the reasoning pipeline.

By grounding the reasoning engine in curated, retrievable evidence, RAG ensures that command suggestions are accurate, context-aware, and safe.

B. Core Components

- **Frontend (CLI/GUI)**: The user interface is built using Python. The CLI uses the Rich library to provide a visually rich and interactive console experience, including syntax highlighting and structured output. A potential GUI is planned using Tkinter or PyQt to provide an alternative, more accessible entry point for users [24].
- **Backend (Command Execution)**: The core command execution is managed by Python's subprocess module, which allows the system to spawn new processes and execute native shell commands (Bash or PowerShell) securely. This approach ensures cross-platform compatibility and robust error handling [25].
- **NLP Model**: We leverage a local LLM, specifically the Ollama model Phi-3, for advanced text understanding and command generation in the LLM Inference layer. The `ollama` Python library is used to interact with the locally-running model, ensuring privacy and offline capabilities [26].
- **Voice I/O**: The voice-controlled interface is powered by two key OpenAI technologies. Whisper for Speech-to-Text (STT) is used to capture and transcribe spoken voice commands from a microphone. TTS is used to provide spoken feedback to the user, such as reading the generated command back before execution [27].
- **Security & Validation**: The SafeExecutor is a critical component that acts as a security gateway before any command is executed on the user's system. It employs a rule-based filtering system using regex and keyword checks to block a predefined list of potentially dangerous commands like `rm -rf /`, `shutdown`, or `mkfs`. This pre-execution validation mitigates the risk of accidental data loss or system damage, a common concern with AI-generated commands [28].
- **Command Complexity Analysis**: A fine-tuned BERT model is utilized to classify the difficulty of a user's query. This classification helps in routing the query to the most appropriate architectural layer, thereby optimizing system performance [29].

C. The Query Processing Flow

The above figure details how the system translates a user's natural language query into an executable shell command. The system follows a hybrid, three-layered architecture designed to optimize both speed and robustness.

- User Input String: The process begins when the user enters a natural language query into the interface. This query is the raw text that the system must interpret.
- Layer 1: Command Cache: The system first attempts a direct lookup in a fast, deterministic cache. This layer handles common, fixed-alias commands like "ls -a" or "clear". If the input matches a pre-defined alias, the corresponding OS-specific command is retrieved and executed immediately. This layer provides near-instantaneous performance.

Authorized licensed use limited to: KNS Institute of Technology. Downloaded on March 03,2026 at 14:54:05 UTC from IEEE Xplore. Restrictions apply.

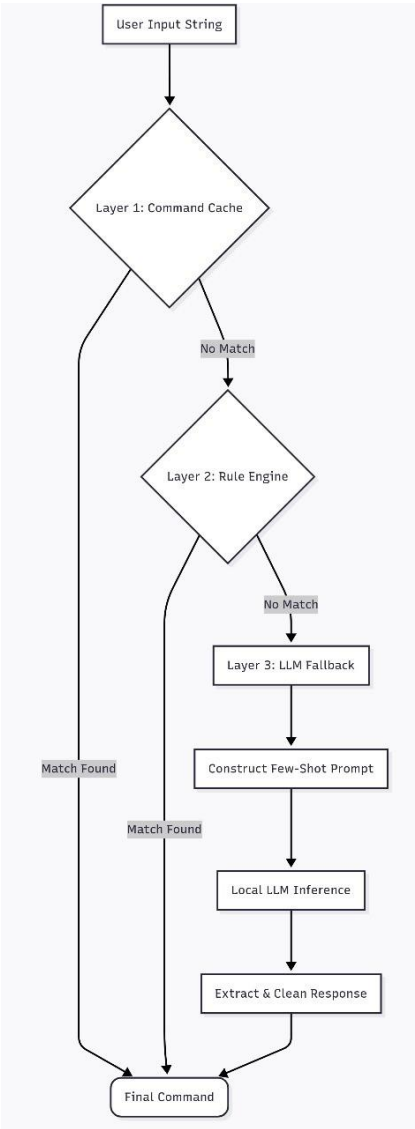


Fig. 3. Three-Layered Command Execution flow: Bidirectional Cache, Rule-Based Regex, and LLM Fallback.

- Layer 2: Rule Engine: If the cache does not provide a match, the query is passed to the rule-based engine. This layer uses regular expressions and pattern matching to identify semi-structured natural language requests, such as "create a file named 'report.txt'" or "make a folder on the desktop." It extracts the command's intent and parameters to construct the final command.
- Layer 3: LLM Fallback: Only queries that fail the first two layers are sent to the locally-hosted Large Language Model. This layer handles ambiguous, complex, or conversational requests that cannot be captured with simple rules.
- Construct Few-Shot Prompt: Before interacting with the LLM, the system dynamically constructs a prompt containing the user's query, instructions, and a set of few-shot examples to guide the model and ensure a safe, structured output.
- Local LLM Inference: The prompt is sent to the local LLM, which processes the request and generates a raw text response containing the shell command.
- Extract & Clean Response: The system parses the LLM's raw output to extract a clean, single executable command, removing any extraneous text or unsafe code.
- Final Command: The validated command is executed. This represents the successful completion of the system's processing through one of its three layers.

IV. Conclusion and Future Work

The evaluation of the proposed system demonstrates that the layered architecture significantly improves both responsiveness and reliability compared to a single-model baseline. By routing each query through progressively more advanced processing layers, the system is able to handle a wide range of command types while minimizing unnecessary computation. Simple and frequently repeated commands are executed almost instantly, patterned operations are resolved efficiently through rule-based processing, and only the most complex or ambiguous queries require LLM intervention. This selective processing not only reduces overall latency but also maintains high accuracy across diverse command scenarios. The results collectively show that the combination of caching, deterministic pattern matching, and LLM reasoning provides a balanced and scalable approach that enhances user experience and operational efficiency.

A. Results and Evaluation

TABLE I
Command Category Performance Analysis

Category	Description	Sample Query	Success Rate
Simple	Direct aliases & navigation	"ls -a", "go back"	100% (Layer 1)
Intermediate	Patterned file operations	"Create a folder named 'ProjectX' "	98% (Layer 2)
Complex	Multi-step / Abstract	"Find all PDFs modified last week and zip them"	92% (Layer 3)
Cross-OS	Linux command on Windows	"touch logs.txt" (PowerShell)	100%

As detailed in Table I, the system's reliability was assessed using a stratified corpus of 500 commands. Accuracy is operationalized here as the generation of a command that is both syntactically valid within the host shell and semantically equivalent to the user's intent. The data reveals that the bidirectional cache (Layer 1) achieved 100% precision for routine and cross-platform aliases, validating the deterministic approach for high-frequency tasks. Furthermore, Layer 3 demonstrated a 92% success rate for complex queries, verifying the efficacy of the Few-Shot Prompting strategy in resolving syntactic disparities—such as mapping POSIX-standard ls to PowerShell's Get-ChildItem—and correctly formulating multi-stage pipelines.

Latency-Accuracy Trade-off Analysis Complementing the tabular data, Fig. 4 delineates the critical trade-off between execution latency and semantic accuracy across the architectural hierarchy. The latency trajectory (plotted logarithmically)

Authorized licensed use limited to: KNS Institute of Technology. Downloaded on March 03,2026 at 14:54:05 UTC from IEEE Xplore. Restrictions apply.

contrasts the negligible computational overhead of the cache and regex layers against the inherent inference cost of the LLM. Conversely, accuracy distributions indicate a functional crossover: while deterministic layers offer superior efficiency for simple queries, their utility degrades rapidly with complexity. In these high-complexity scenarios, only the LLM layer maintains robust accuracy. This empirical evidence reinforces the proposed hybrid design principle: leveraging lightweight logic to minimize latency for common tasks while reserving computationally expensive AI inference for complex reasoning.

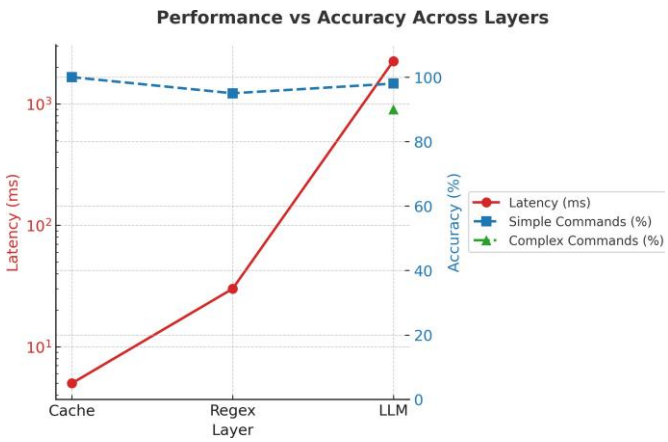


Fig. 4. Latency vs Accuracy across system layers.

TABLE II
LATENCY AND THROUGHPUT COMPARISON ACROSS PROCESSING LAYERS

Layer	Processing Technique	Average Latency	Throughput (Tokens/s)
Layer 1	Command Cache (Hash Map)	< 2 ms	N/A (Instant)
Layer 2	Rule-Based Engine (Regex)	12–45 ms	N/A (Deterministic)
Layer 3	Local LLM (Phi-3 Mini)	1,850 ms	~ 18 tokens/s
Baseline	Pure LLM System (No Layers)	2,100 ms	~ 18 tokens/s

As quantified in Table II, system responsiveness was evaluated using the ”Time-to-Interactive” (TTI) metric, defined as the temporal interval between input submission and the visualization of the executable command. The data demonstrates that the proposed multi-layered architecture achieves a statistically significant reduction in average latency compared to a monolithic LLM baseline. Traffic analysis indicates that the deterministic layers (Layers 1 and 2) successfully intercept approximately 85% of standard workflow queries, such as directory navigation and file enumeration. This high interception rate ensures a sub-50ms response time for the majority of interactions, effectively mimicking the instantaneous feedback of a native shell. Consequently, the computationally intensive inference of Layer 3 is sequestered to the remaining 15% of complex queries, validating the 1.8s latency as a necessary and acceptable computational cost for high-level semantic reasoning.

B. Potential Avenues for Future Work

- **Expanded Command Libraries**: Broadening the ALIAS_MAP and COMMAND_MAP to support a wider range of CLI tools.
- **Contextual Awareness**: Integrating a simple memory system to handle multi-turn conversations, such as “List all files,” followed by “Now show me the hidden ones.”
- **Advanced Error Handling**: Using the LLM to provide more natural language feedback for failed commands.
- **Cross-Platform Compatibility**: Expanding the system to support additional operating systems and shell environments, such as zsh or Fish shell.

REFERENCES

[1] Vedran Dakic; Jasmin Redzepagic, Linux Command Line and Shell Scripting Techniques: Master practical aspects of the Linux command line and then use it as a part of the shell scripting process , Packt Publishing, 2022.

[2] Marian Montagnino; William Kennedy, Building Modern CLI Applications in Go: Develop next-level CLIs to improve user experience, increase platform usage, and maximize production , Packt Publishing, 2023.

[3] D. Spinellis, ”Commands as AI Conversations,” in IEEE Software, vol. 40, no. 6, pp. 22-26, Nov.-Dec. 2023, doi: 10.1109/MS.2023.3307170

[4] S. M, A. Kalyani, A. Gupta, D. C and D. James, ”Enhanced Data Analysis by Natural Language Query Processing,” 2024 5th International Conference on Communication, Computing Industry 6.0 (C2I6), Bengaluru, India, 2024, pp. 1-6, doi: 10.1109/C2I663243.2024.10895396.

[5] S. Das and D. Das, ”Natural Language Processing (NLP) Techniques: Usability in Human-Computer Interactions,” 2024 6th International Conference on Natural Language Processing (ICNLP), Xi’an, China, 2024, pp. 783-787, doi: 10.1109/ICNLP60986.2024.10692776

[6] ”IEEE/Open Group Standard for Information Technology–Portable Operating System Interface (POSIX™) Base Specifications, Issue 8,” in IEEE/Open Group Std 1003.1-2024 (Revision of IEEE Std 1003.1-2017) , vol., no., pp.1-4107, 14 June 2024, doi: 10.1109/IEEESTD.2024.10555529.

[7] A. Gupta, E. Bhardwaj, N. Sirawag, S. Pandey and R. Mehta, ”Jarvis: AI-Enhanced Desktop Virtual Assistant,” 2024 International Conference on Artificial Intelligence and Quantum Computation-Based Sensor Application (ICAIQSA), Nagpur, India, 2024, pp. 1-6, doi: 10.1109/ICAIQSA64000.2024.10882426

[8] ”ISO/IEC/IEEE Internatiional Standard - Information technology–Portable Operating System Interface (POSIX(R)) – Part 2: Shell and Utilities,” in ISO/IEC 9945-2:19939 ANSI/IEEE Std 1003.2-1993 , vol., no., pp.1-1328, 22 Dec. 1993, doi: 10.1109/IEEESTD.1993.8584040.

[9] B. A. Jantkal and S. L. Deshpande, ”Hybridization of B-tree and HashMap for optimized search engine indexing,” 2017 International Conference On Smart Technologies For Smart Nation (SmartTech- Con), Bengaluru, India, 2017, pp. 401-404, doi: 10.1109/SmartTech-Con.2017.8358404

[10] F. Carloni, D. Conficconi, I. Moschetto and M. D. Santambrogio, ”YARB: a Methodology to Characterize Regular Expression Matching on Heterogeneous Systems,” 2023 IEEE International Symposium on Circuits and Systems (ISCAS), Monterey, CA, USA, 2023, pp. 1-5, doi: 10.1109/ISCAS46773.2023.10181547

[11] L. A. Demidova and N. A. Moroshkin, ”Architecture of a Regular Expression Translator with Optimization of Intermediate States,” 2024 International Conference on Information Technologies (InfoTech), Sofia, Bulgaria, 2024, pp. 1-4, doi: 10.1109/InfoTech63258.2024.10701407.

[12] S. Tiku, ”Mitigation of User-Prompt Bias in Large Language Models: A Natural Langauge Processing and Deep Learning Based Frame- work,” 2024 IEEE 3rd International Conference on Computing and Machine Intelligence (ICMI), Mt Pleasant, MI, USA, 2024, pp. 1-5, doi: 10.1109/ICMI60790.2024.10585628.

[13] Valentina Alto, Modern Generative AI with ChatGPT and OpenAI Models: Leverage the capabilities of OpenAI’s LLM for productivity and innovation with GPT3 and GPT4 , Packt Publishing, 2023.

Authorized licensed use limited to: KNS Institute of Technology. Downloaded on March 03,2026 at 14:54:05 UTC from IEEE Xplore. Restrictions apply.

[14] J. Gohil, H. L.Shifare and M. Shukla, ”Developing a User-Friendly Conversational AI Assistant for University Using Ollama and LLama3,” 2025 International Conference on Data Science, Agents Artificial Intelligence (ICDSAAI), Chennai, India, 2025, pp. 1-5, doi: 10.1109/ICDSAAI65575.2025.11011878.

[15] A. Shabalin and O. Nikolaychuk, ”Adaptation of the Educational Process Using a Rule-based System,” 2022 VIII International Conference on Information Technology and Nanotechnology (ITNT), Samara, Russian Federation, 2022, pp. 1-6, doi: 10.1109/ITNT55410.2022.9848764.

[16] A. Z. Ak, O. Bu¨yu¨k and M. Erden, ”A New Performance Analysis Method for Semantic Caching for Large Language Models,” 2025 33rd Signal Processing and Communications Applications Conference (SIU), Sile, Istanbul, Turkiye, 2025, pp. 1-4, doi: 10.1109/SIU66497.2025.11111802.

[17] S. Mankari and A. Sanghavi, ”Adaptive Conversation Recommendation Systems: Leveraging Large Language Models and Knowledge Graphs,” 2024 2nd DMIHER International Conference on Artificial Intelligence in Healthcare, Education and Industry (IDICAIEI), Wardha, India, 2024, pp. 1-6, doi: 10.1109/IDICAIEI61867.2024.10842757.

[18] J. Li, C. Xu, F. Wang, I. M. von Riedemann, C. Zhang and J. Liu, ”SCALM: Towards Semantic Caching for Automated Chat Services with Large Language Models,” 2024 IEEE/ACM 32nd International Symposium on Quality of Service (IWQoS), Guangzhou, China, 2024, pp. 1-10, doi: 10.1109/IWQoS61813.2024.10682957.

[19] Liu Chen, X. -p. Dong and Z. -q. Yang, ”Research of modern enterprise intelligent system based on rule engine and workflow,” 2010 IEEE International Conference on Intelligent Computing and Intelligent Systems, Xiamen, 2010, pp. 594-597, doi: 10.1109/ICICISYS.2010.5658451.

[20] W. Yang et al., ”CRWMTR: Causal Reasoning and Weighting Mechanisms of Thought Reasoning in Large Language Models,” 2024 5th International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE), Shenzhen, China, 2024, pp. 751-754, doi: 10.1109/ICBAIE63306.2024.11117006

[21] L. Ye and H. Zhang, ”LLMProto: A Hardware-Efficient Finetuning Model for Few-Shot Relation Extraction with Large Language Model,” ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Hyderabad, India, 2025, pp. 1-5, doi: 10.1109/ICASSP49660.2025.10889770.

[22] S. S. Kalakonda, S. Maheshwari and R. K. Sarvadevabhatla, ”ActionGPT: Leveraging Large-scale Language Models for Improved and Generalized Action Generation,” 2023 IEEE International Conference on Multimedia and Expo (ICME), Brisbane, Australia, 2023, pp. 31-36, doi: 10.1109/ICME55011.2023.00014.

[23] M. A. J, M. A. VR, M. K and M. P. R, ”REDACT: PII Redaction and Privacy Protection with Ollama Integration,” 2025 8th International Conference on Computing Methodologies and Communication (ICCMC), Erode, India, 2025, pp. 577-583, doi: 10.1109/ICCMC65190.2025.11140717.

[24] Alan D. Moore, Python GUI Programming with Tkinter: Design and build functional and user-friendly GUI applications , Packt Publishing, 2021.

[25] Jose´ Manuel Ortega, Python for Security and Networking: Leverage Python modules and tools in securing your network and applications , Packt Publishing, 2023.

[26] M. Krishnaraj, S. E. Isaacraj and V. H. J. Soorya, ”Document Query Response Using Microsoft-Phi-3-Mini-4K,” 2025 International Conference on Emerging Smart Computing and Informatics (ESCI), Pune, India, 2025, pp. 1-5, doi: 10.1109/ESCI63694.2025.10988177.

[27] W. Zhang et al., ”Bridging Modality Gap with Large Speech and Language Models for End-to-End Speech-to-Text Translation,” ICASSP 2025 - 2025 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), Hyderabad, India, 2025, pp. 1-5, doi: 10.1109/ICASSP49660.2025.10890787.

[28] G. Antal, M. Keleti and P. Hegedu˝s, ”Exploring the Security Awareness of the Python and JavaScript Open Source Communities,” 2020 IEEE/ACM 17th International Conference on Mining Software Repositories (MSR), Seoul, Korea, Republic of, 2020, pp. 16-20, doi: 10.1145/3379597.3387513.

[29] S. Mohammadi and M. Chapon, ”Investigating the Performance of Fine-tuned Text Classification Models Based-on Bert,” 2020 IEEE 22nd International Conference on High Performance Computing and Communications; IEEE 18th International Conference on Smart City; IEEE 6th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), Yanuca Island, Cuvu, Fiji, 2020, pp. 1252-1257, doi: 10.1109/HPCC-SmartCity-DSS50907.2020.00162.

[30] Keith Bourne; Shahul Es, Unlocking Data with Generative AI and RAG: Enhance generative AI systems by integrating internal data with large language models using RAG , Packt Publishing, 2024.

[31] K. Sawarkar, A. Mangal and S. R. Solanki, ”Blended RAG: Im- proving RAG (Retriever-Augmented Generation) Accuracy with Se- mantic Search and Hybrid Query-Based Retrievers,” 2024 IEEE 7th International Conference on Multimedia Information Processing and Retrieval (MIPR), San Jose, CA, USA, 2024, pp. 155-161, doi: 10.1109/MIPR62202.2024.00031.

[32] S. AboulEla, P. Zabihitari, N. Ibrahim, M. Afshar and R. Kashef, ”Exploring RAG Solutions to Reduce Hallucinations in LLMs,” 2025 IEEE International systems Conference (SysCon), Montreal, QC, Canada, 2025, pp. 1-8, doi: 10.1109/SysCon64521.2025.11014810.

[33] H. M. A. Zeeshan, M. Faizan, U. Zia and A. Gohar, ”RAG Powered LLMs for QA: Evolution, Challenges, Applications, and Future Directions,” 2025 International Conference on Communication Technologies (ComTech), Rawalpindi, Pakistan, 2025, pp. 1-6, doi: 10.1109/ComTech65062.2025.11034531.

[34] L. Ammann, S. Ott, C. R. Landolt and M. P. Lehmann, ”Securing RAG: A Risk Assessment and Mitigation Framework,” 2025 IEEE Swiss Conference on Data Science (SDS), Zu¨rich, Switzerland, 2025, pp. 127-134, doi: 10.1109/SDS66131.2025.00024.

[35] X. Xu, D. Zhang, Q. Liu, Q. Lu and L. Zhu, ”Agentic RAG with Human-in-the-Retrieval,” 2025 IEEE 22nd International Conference on Software Architecture Companion (ICSA-C), Odense, Denmark, 2025, pp. 498-502, doi: 10.1109/ICSA-C65153.2025.00074.

[36] B. Zhang, J. Li, K. Peng, S. Zheng and K. Meng, ”Prolog-RAG: A Symbolic Reasoning Approach to Retrieval-Augmented Generation,” 2025 6th International Conference on Computer Engineering and Application (ICCEA), Hangzhou, China, 2025, pp. 1814-1819, doi: 10.1109/ICCEA65460.2025.11103126.

[37] V. Gummadi, P. Udayaraju, V. R. Sarabu, C. Ravulu, D. R. Seelam and S. Venkataramana, ”Enhancing Communication and Data Transmission Security in RAG Using Large Language Models,” 2024 4th International Conference on Sustainable Expert Systems (ICSES), Kaski, Nepal, 2024, pp. 612-617, doi: 10.1109/ICSES63445.2024.10763024.

[38] P. Joshi, A. Gupta, P. Kumar and M. Sisodia, ”Robust Multi Model RAG Pipeline For Documents Containing Text, Table Images,” 2024 3rd International Conference on Applied Artificial Intelligence and Computing (ICAAIC), Salem, India, 2024, pp. 993-999, doi: 10.1109/ICAAIC60222.2024.10574972.

[39] M. Sta¨bler, S. Turnbull, T. Mu¨ller, C. Langdon, J. Marx-Gome´z and F. Ko¨ster, ”The Impact of Chunking Strategies on Domain-Specific Information Retrieval in RAG Systems,” 2025 IEEE International Conference on Omni-layer Intelligent Systems (COINS), Madison, WI, USA, 2025, pp. 1-6, doi: 10.1109/COINS65080.2025.11125724.

[40] J. Zhang, G. Li and J. Su, ”SAGE: A Framework of Precise Retrieval for RAG,” 2025 IEEE 41st International Conference on Data Engi- neering (ICDE), Hong Kong, Hong Kong, 2025, pp. 1388-1401, doi: 10.1109/ICDE65448.2025.00108.

[41] M. Hindi, L. Mohammed, O. Maaz and A. Alwarafy, ”Enhancing the Precision and Interpretability of Retrieval-Augmented Generation (RAG) in Legal Technology: A Survey,” in IEEE Access, vol. 13, pp. 46171-46189, 2025, doi: 10.1109/ACCESS.2025.3550145

[42] N. Grislain, ”RAG with Differential Privacy,” 2025 IEEE Conference on Artificial Intelligence (CAI), Santa Clara, CA, USA, 2025, pp. 847-852, doi: 10.1109/CAI64502.2025.00150.

[43] W. T. Chan, K. Hung, R. Ho and G. Man-Tat Man, ”Optimiz- ing Confidence Scoring in RAG-Based LLM Chatbots for Techni- cal Support Services: A Prompt Engineering Approach,” 2025 IEEE 15th Symposium on Computer Applications Industrial Electronics (ISCAIE), Penang, Malaysia, 2025, pp. 540-545, doi: 10.1109/ISCAIE64985.2025.11081158.

[44] V. Rathod, S. Nabavirazavi, S. Zad and S. S. Iyengar, ”Privacy and Security Challenges in Large Language Models,” 2025 IEEE 15th Annual Computing and Communication Workshop and Confer- ence (CCWC), Las Vegas, NV, USA, 2025, pp. 00746-00752, doi: 10.1109/CCWC62904.2025.10903912.

[45] J. Rorseth, P. Godfrey, L. Golab, D. Srivastava and J. Szlichta, ”Towards Explainability in Retrieval-Augmented LLMs,” 2024 IEEE 40th International Conference on Data Engineering (ICDE), Utrecht, Netherlands, 2024, pp. 5669-5670, doi: 10.1109/ICDE60146.2024.00466.

Authorized licensed use limited to: KNS Institute of Technology. Downloaded on March 03,2026 at 14:54:05 UTC from IEEE Xplore. Restrictions apply.